

Exercício Programa 1

MAC0422 - Sistemas Operacionais (2026)

Implementando um Shell “imesh” e um Simulador de Processos



Apresentação de Slides

Roteiro

3 - Discussão sobre determinismo

4 - Considerações sobre critérios do escalonador de prioridades

5 - Sobre os testes realizados

6 e 7 - Gráficos com análise de desempenho

8 - Descrição breve dos traces

9 - Configurações das máquinas e conclusões

10 - Referências utilizadas

Determinístico?

QUASE!



Houve pequenas oscilações, mas *somente nos escalonadores preemptivos*. Não há controle sobre o comportamento das threads e a competição por recursos por parte delas, principalmente dentro das filas - no decorrer da execução. A própria ordem do trace - estabelecida antes da execução - pode afetar também a inserção nas filas quando o tempo de chegada é o mesmo.

Com os desvios padrão pequenos, mas não nulos, podemos dizer que **não**.

Escalonador de Prioridades

- ❑ 5 classes de prioridades (inspirado em *Tanenbaum, Bos, p. 111 - 4ª edição*, onde há round-robin em cada classe, e só executa processos da classe mais abaixo se a atual estiver vazia).
- ❑ Classes da maior para a menor: 4: deadline crítico (está de 0 a 5 segundos do mesmo, se começar a rodar agora); 3: deadline médio (5 a 10 segundos); 2: deadline distante (10 a 15 segundos) 1: deadline bem distante (mais de 15 segundos); 0: deadline impossível de cumprir (perdeu o prazo, vai pro final). Critério de justiça com foco em cumprir deadlines.
- ❑ Quantum FIXO e o IGUAL ao do round-robin (2 segundos)!

Desvantagens: o escalonador perde um pouco de generalidade; o estabelecimento das prioridades pode ser menos justo a depender da entrada, gerando *starvation* nas menores classes.

Vantagens: comparação mais justa com o round-robin; t_0 's são sempre inteiros na nossa simulação, logo o quantum usado traz mais justiça e evita overhead desnecessário.

Automação dos Testes

```
#!/bin/bash
```

```
read -p "Digite a máquina sendo testada [A|B]: " maquina
```

```
echo "iniciando fase de testes"
```

```
for i in {1..30}; do
```

```
  for j in {1..3}; do
```

```
    echo "rodando traces com escalonador ${j}!"
```

```
    ./ep1 ${j} entrada-esperado.txt
```

```
saida-esperado-sched=${j}-maquina=${maquina}-teste=${i}.txt
```

```
    ./ep1 ${j} entrada-inesperado.txt
```

```
saida-inesperado-sched=${j}-maquina=${maquina}-teste=${i}.txt
```

```
  done
```

```
  echo "teste ${i} realizado"
```

```
done
```

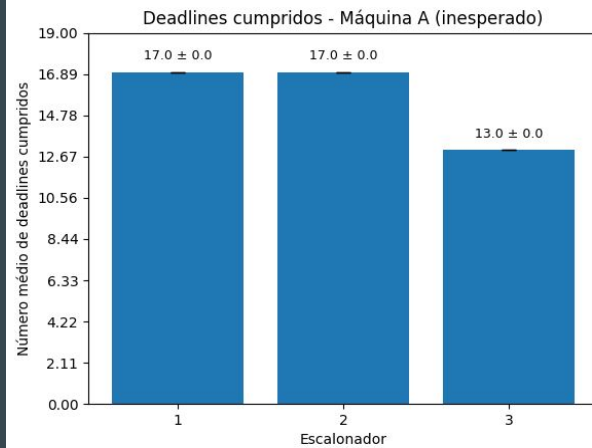
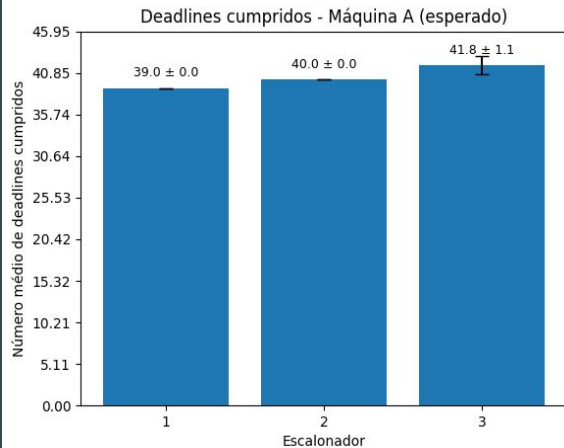
Tivemos 360 saídas no total, levando algumas horas para realizar todos os testes.

Em todo o tempo, apenas o terminal com a execução do EP esteve aberto, para reproduzir as condições de simulação mais próximas possíveis.

Cumprimento dos Deadlines

1 = SJF, 2 = Round-Robin, 3 = Prioridades

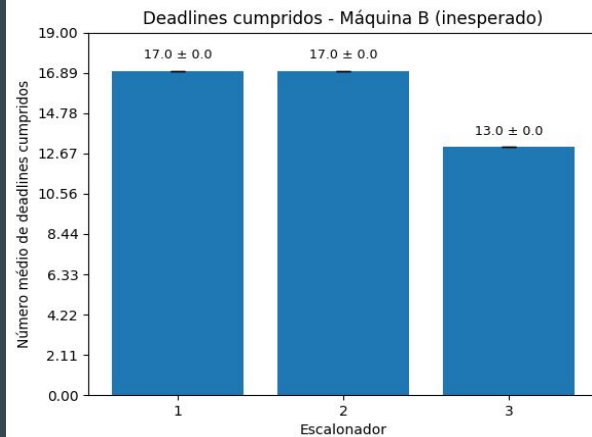
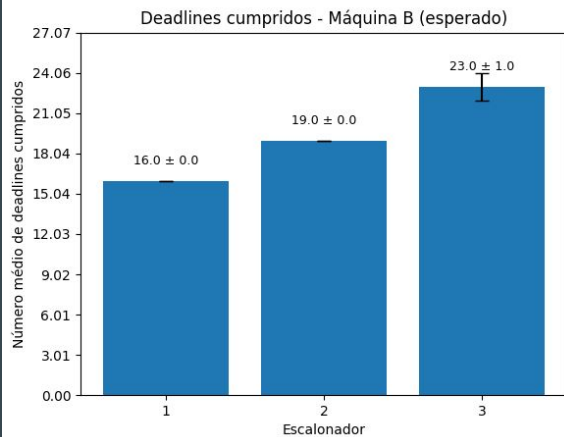
Máquina A
(12 núcleos)



*esperado
tem 44
processos

*inesperado
tem 47
processos

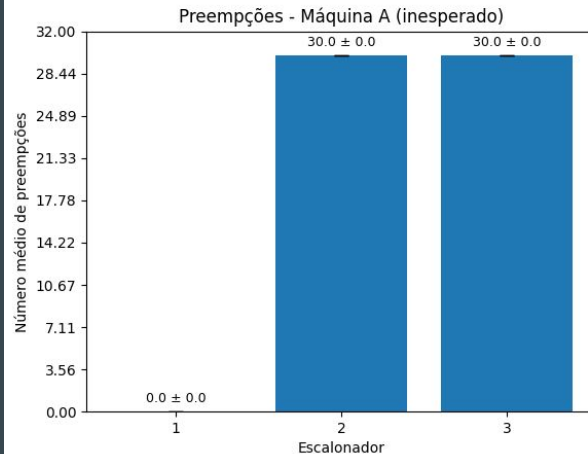
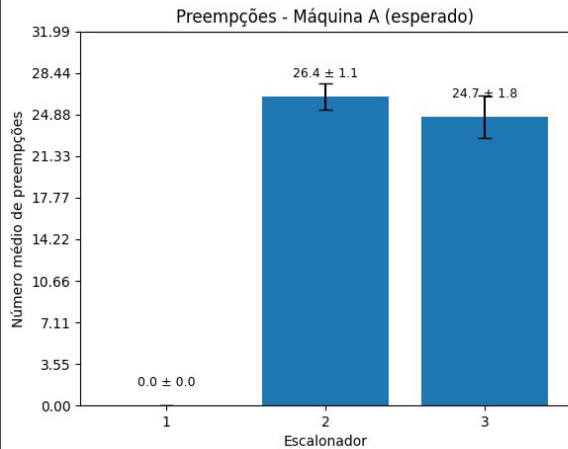
Máquina B
(4 núcleos)



Quantidade de Preempções

1 = SJF, 2 = Round-Robin, 3 = Prioridades

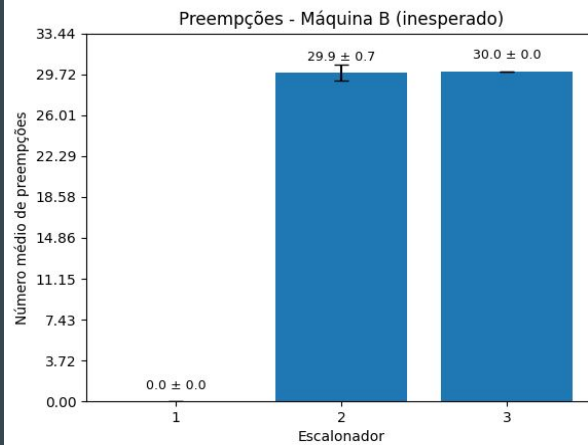
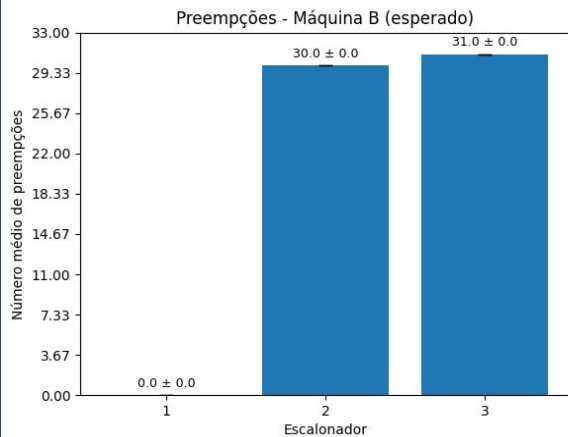
Máquina A
(12 núcleos)



*esperado
tem 44
processos

*inesperado
tem 47
processos

Máquina B
(4 núcleos)



Entradas: Resultados Esperados e Inesperados

- ❑ Ambos arquivos de trace têm 120 segundos como soma dos tempos de processo.
- ❑ entrada-esperado.txt tem 44 processos, entrada-inesperado.txt tem 47 processos.
- ❑ O desenvolvimento foi realizado na máquina A com 12 núcleos, dificultando a diferenciação e a geração de bons traces (solução: tentativa e erro), por isso tantos processos.
- ❑ Cumprimento de deadlines dentro do intervalo de confiança.
- ❑ Diferenças apertadas em mais núcleos: possivelmente as preempções do RR o fazem perder algum tempo realizando-as, enquanto o shortest job “engata a execução” de forma direta. O de prioridades otimiza as preempções e foi desenhado para cumprir deadlines - por isso foi, em geral, superior.
- ❑ ‘entrada-esperado.txt’ reproduz um cenário em que o SJF perde tempo com processos curtos de prazo relaxado, e o RR traz justiça para processos longos. O de prioridades até perde alguns prazos apertados de processos curtos cumpridos pelos outros, mas compensa “salvando” aqueles mais longos.
- ❑ ‘entrada-inesperado.txt’ traz uma situação atípica onde todos processos têm mesmo t_0 , o escalonador de prioridades é “enganado” ao escalonar: tenta salvar processos de deadline apertado, perde o prazo, e ainda causa *starvation* em 4 deles com prioridade intermediária
=> Cumpre 4 prazos a menos.

Configurações Utilizadas e Conclusões

Máquina A (12 cores)

Intel Core i5-13420H, 13ª gen.

Fedora Linux 43 (Workstation Edition)

(12 núcleos lógicos, 8 físicos)



Máquina B (4 cores)

Intel Core i3-8100, 8ª gen.

Debian GNU/Linux 13 (trixie)



- ❑ Podemos gerar traces que beneficiem cada escalonador de alguma forma, pensando na arquitetura deles, e também explorando suas “falhas”.
- ❑ Uma máquina com mais núcleos tem resultados esperados, em termos de executar mais processos em um mesmo intervalo de tempo, independente do escalonador.
- ❑ Com menos núcleos ocorrem mais preempções, também esperado já que há maior competição por recursos.
- ❑ Em um cenário com limite de duração de 120 segundos, o SJF performa bem, mas a natureza preemptiva dos outros ainda é mais adequada em sistemas interativos.

Referências

Livros:

Andrew S. Tanenbaum, "Sistemas operacionais modernos", Pearson Education do Brasil Pearson, 2010.

Chatbots (sem código gerado, apenas discussão):

<https://claude.ai/share/56892fcf-0b75-4c40-9a21-3e86b1a8b482>

Manpages (syscalls):

<https://man7.org/linux/man-pages/man3/getcwd.3.html>

<https://man7.org/linux/man-pages/man2/time.2.html>

<https://man7.org/linux/man-pages/man1/kill.1.html>

<https://man7.org/linux/man-pages/man3/sscanf.3.html>

<https://man7.org/linux/man-pages/man2/nanosleep.2.html>

https://man7.org/linux/man-pages/man3/pthread_setaffinity_np.3.html

<https://unix.stackexchange.com/questions/317492/list-of-kill-signals>

Utilitários em C usados:

https://linux.die.net/man/3/strtok_r

<https://www.geeksforgeeks.org/c/understanding-volatile-qualifier-in-c/>

<https://www.geeksforgeeks.org/c/queue-using-linked-list-in-c/>

Evitando Deadlocks:

<https://www.codequoi.com/en/threads-mutexes-and-concurrent-programming-in-c/#the-danger-of-data-races>